EXP 10	A PYTHON PROGRAM TO PERFORM PERT ANALYSIS

AIM:

To perform Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python.

PROCEDURE:

1. INSTALL REQUIRED LIBRARIES:
   •Make sure you have pandas, networkx, matplotlib, and numpy installed. •You can install them using pip if not already installed. -bash -Copy code -pip install pandas networkx matplotlib numpy
2. DEFINE THE PROBLEM:
   •Identify the tasks, their optimistic, most likely, and pessimistic durations, and dependencies.
3. SET UP THE PROBLEM IN PYTHON:
   •Use NetworkX to represent the project tasks and their dependencies
   •Calculate the expected duration and variance for each task.
   •Use topological sort to identify the critical path and perform PERT analysis.
4. RUN THE CODE:
   •Execute the Python code to get the critical path, project duration, and associated uncertainty.
5. ANALYZE THE OUTPUT:
   •Interpret the results and validate the solution against the project schedule.

PROBLEM 1:
A project consists of the following activities with three time estimates:

Activity	Optimistic (o)	Most Likely (m)	Pessimistic (p)	Predecessors
A	1	2	3	—

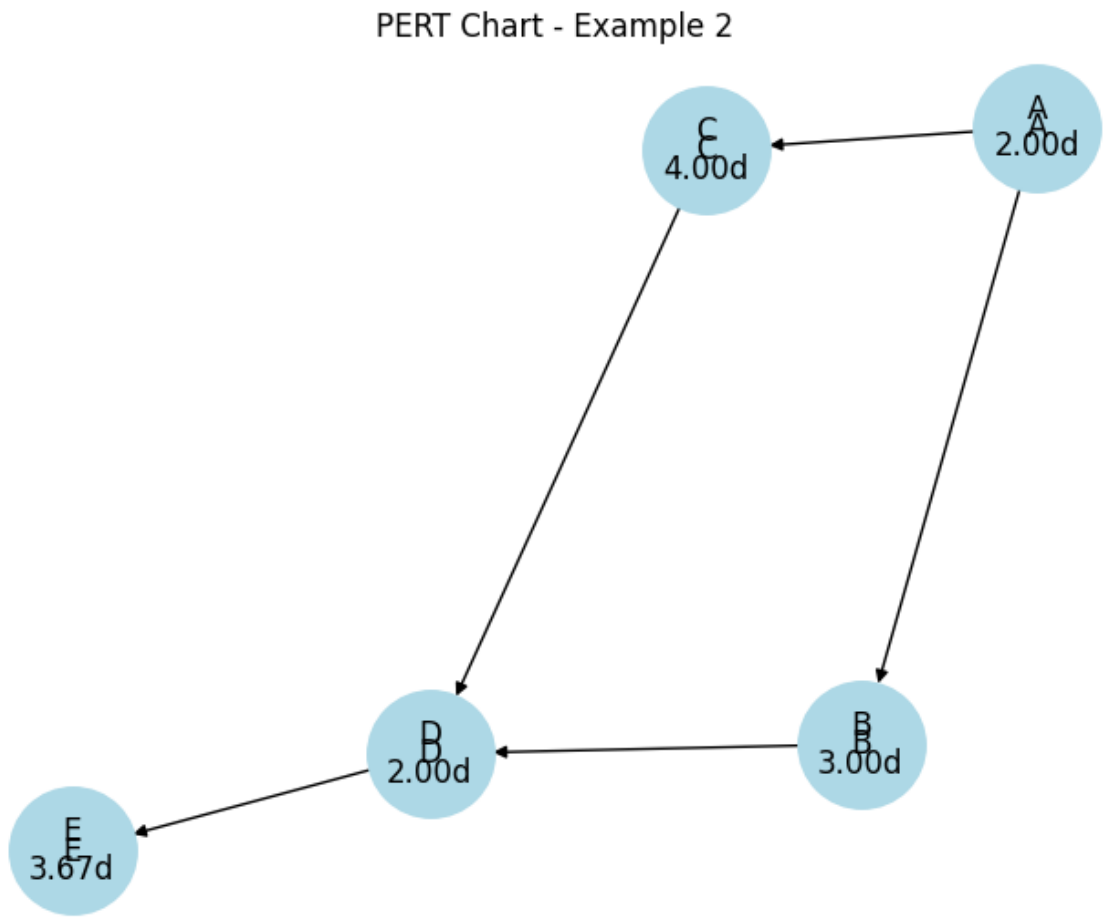
Activity	Optimistic (o)	Most Likely (m)	Pessimistic (p)	Predecessors
B	1	3	5	A
C	2	4	6	A
D	1	2	3	B, C
E	2	3	8	D

CODE:

```
import networkx as nx
import matplotlib.pyplot as plt
def expected_time(o, m, p):
    return (o + 4*m + p) / 6
def pert_example2():
    activities = {
        'A': (1, 2, 3),
        'B': (1, 3, 5),
        'C': (2, 4, 6),
        'D': (1, 2, 3),
        'E': (2, 3, 8)
    }
    dependencies = {
        'A': [],
        'B': ['A'],
        'C': ['A'],
        'D': ['B', 'C'],
        'E': ['D']
    }
    durations = {k: expected_time(*v) for k, v in activities.items()}
    G = nx.DiGraph()
    for task, duration in durations.items():
        G.add_node(task, duration=duration)
    for task, deps in dependencies.items():
        for dep in deps:
            G.add_edge(dep, task)
    earliest_start = {}
```

```
    for node in nx.topological_sort(G):
        es = 0
        for pred in G.predecessors(node):
            es = max(es, earliest_start[pred] + G.nodes[pred]['duration'])
        earliest_start[node] = es
    total_duration = max(
        earliest_start[n] + G.nodes[n]['duration']
        for n in G.nodes
    )
print("Expected Project Duration:", total_duration)
    pos = nx.spring_layout(G)
    nx.draw(G, pos, with_labels=True,
            node_size=2500,
            node_color='lightblue')
    labels = {
        node: f"{node}\n{durations[node]:.2f}d"
        for node in G.nodes
    }
    nx.draw_networkx_labels(G, pos, labels=labels)
    plt.title("PERT Chart - Example 2")
    plt.show()
pert_example2()
```

OUTPUT:
Expected Project Duration: 11.666666666666666



PROBLEM 2: PERK ANALYSIS
A project has the following activities:

Activity	Optimistic (o)	Most Likely (m)	Pessimistic (p)	Predecessors
A	2	4	6	—
B	1	2	3	A
C	2	3	4	A
D	3	5	7	B
E	4	6	8	C
F	2	4	6	D, E

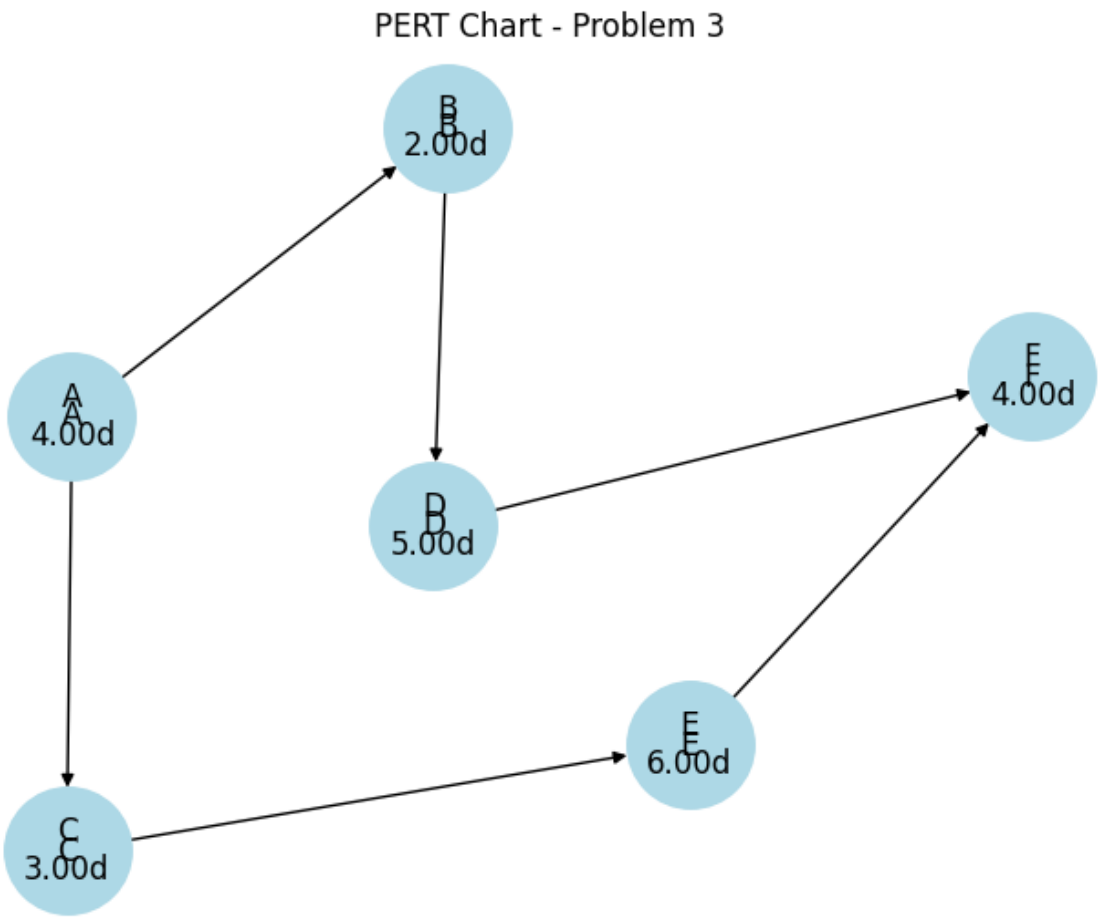
CODE:

```
import networkx as nx
import matplotlib.pyplot as plt
def expected_time(o, m, p):
    return (o + 4*m + p) / 6
def pert_problem3():
    activities = {
        'A': (2, 4, 6),
        'B': (1, 2, 3),
        'C': (2, 3, 4),
        'D': (3, 5, 7),
        'E': (4, 6, 8),
        'F': (2, 4, 6)
    }
    dependencies = {
        'A': [],
        'B': ['A'],
        'C': ['A'],
        'D': ['B'],
        'E': ['C'],
        'F': ['D', 'E']
    }
    durations = {k: expected_time(*v) for k, v in activities.items()}
    G = nx.DiGraph()
    for task, duration in durations.items():
        G.add_node(task, duration=duration)
    for task, deps in dependencies.items():
        for dep in deps:
            G.add_edge(dep, task)
    earliest_start = {}
    for node in nx.topological_sort(G):
        es = 0
        for pred in G.predecessors(node):
            es = max(es, earliest_start[pred] + G.nodes[pred]['duration'])
        earliest_start[node] = es
    total_duration = max(
        earliest_start[n] + G.nodes[n]['duration'] for n in G.nodes
    )
    print("Expected Project Duration:", total_duration)
    pos = nx.spring_layout(G)
    nx.draw(G, pos, with_labels=True,
```

```
            node_size=2500,
            node_color='lightblue')
    labels = {
        node: f"{node}\n{durations[node]:.2f}d"
        for node in G.nodes
    }
    nx.draw_networkx_labels(G, pos, labels=labels)
    plt.title("PERT Chart - Problem 3")
    plt.show()
pert_problem3()
```

OUTPUT:

Expected Project Duration: 17.0



RESULT:
Thus Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python has been successfully implemented.